

EXERCISE-1

POOSA VISHNUVARDHAN

90119953

LEVEL 1:

Q1. File-System Problems

The two problems are **data redundancy** and **data inconsistency**. Data redundancy occurs because the same customer address is stored in multiple spreadsheets, while data inconsistency occurs when the branch updates its copy but the credit card department still has the old address. A database stores the customer information centrally so different departments can use the same record, and a single update can be reflected wherever that data is required. This reduces duplicate copies and prevents different departments from working with conflicting versions of the same address.

Q2. Three Things a DBMS Does That a File System Does Not

1. **Data integrity enforcement:** A DBMS can enforce rules such as unique customer IDs, valid account references and non-negative balances. Without these controls, invalid or inconsistent banking records could be stored.
2. **Concurrent transaction management:** A DBMS controls simultaneous transactions from ATMs, mobile banking and branch systems. Without it, two transactions could update the same account incorrectly and cause an inaccurate balance.
3. **Transaction recovery:** A DBMS maintains transaction logs and recovery mechanisms so committed data can survive failures. Without this, a power failure could leave a transfer partially completed or cause committed banking data to be lost.

Q3. Relational Terms

Formal Relational Term	Everyday Term	ICICI Bank Example
Relation	Table	customer table
Tuple	Row / Record	One customer's complete record
Attribute	Column / Field	customer_id column
Degree	Number of columns	If customer has 9 columns, its degree is 9
Cardinality	Number of rows / Records	If customer has 30 rows, its cardinality is 30

Q4. Classification of Keys

1. customer.customer_id — **Primary Key**
2. account.account_number — **Candidate Key** (assuming it is declared UNIQUE and can uniquely identify an account)
3. account.customer_id — **Foreign Key**

4. account.balance — **None**
5. employee.manager_id — **Foreign Key** if it references employee.employee_id; it represents the employee's manager.
6. branch.ifsc_code — **Candidate Key** (assuming it is declared UNIQUE and identifies a branch)

Q5. OLTP or OLAP

1. **A customer checks a balance on the ICICI mobile app — OLTP**
Reason: It is a real-time operational transaction involving a current account record.
2. **The risk team calculates non-performing loans by branch for the quarter — OLAP**
Reason: It performs analytical aggregation over a large amount of historical data.
3. **An ATM dispenses 10,000 rupees — OLTP**
Reason: It is a real-time banking transaction that updates an account balance.
4. **A regulator asks for the total value of transactions above 200,000 in the last financial year — OLAP**
Reason: It requires analysis and aggregation of historical transaction data.
5. **A teller opens a new savings account — OLTP**
Reason: It is an operational transaction that creates and updates customer and account records.
6. **Marketing finds the top 100 customers by average monthly balance — OLAP**
Reason: It requires analysis and aggregation across historical account data.

Q6. Running Regulatory Reports Directly on the Core Oracle Database

The reporting query could consume CPU, memory, disk I/O and database connections needed by real-time banking transactions. Large scans and aggregations could also increase response times for customer-facing operations such as ATM withdrawals and mobile banking. Reporting directly against the core system also makes it harder to isolate reporting workloads, schedule heavy queries and maintain a stable operational environment. I would instead offer a **read replica or a separate reporting/analytical database populated from the core system**, so the reporting team gets sufficiently fresh data without putting unnecessary load on the production database.

Q7. ACID Properties and Failures

Property Failure

Atomicity d. Money is debited but never credited

Consistency c. An account exists for a customer who does not

Isolation b. Two ATMs both pay out against the same balance

Durability a. A committed transfer disappears after a power cut

Q8. Write-Ahead Log

The write-ahead log is written to durable storage before the corresponding data page because the log provides the information needed to recover the transaction if the system crashes. If the data

page were written first and the system failed before the log was safely stored, the database might contain a change that cannot be properly recovered, potentially causing corruption or loss of transactional guarantees.

Q9. Data Independence and SELECT *

This breaks **physical data independence at the application interface level** by making the application depend unnecessarily on the complete table structure and column order rather than selecting only the required columns. When the bank adds a `pan_number` column, `SELECT *` starts returning an additional column, which can break application code, APIs, reports or code that assumes a fixed result structure. The safer approach is to explicitly select the required columns.

Q10. Oracle, PostgreSQL, MySQL and Redis

Oracle: Use Oracle for core banking workloads requiring mature enterprise transaction management, high availability and strong data integrity.

PostgreSQL: Use PostgreSQL for applications and services requiring a powerful open-source relational database with strong SQL and transaction support.

MySQL: Use MySQL for suitable high-volume web and application workloads where its ecosystem and operational simplicity are useful.

Redis: Use Redis for very fast in-memory use cases such as caching, sessions, rate limiting and temporary application data.

MODULE 1 | LEVEL 2 — APPLYING SQL

Q11. Database Name, Current User and Server Port

For PostgreSQL:

```
SELECT current_database() AS database_name,  
       current_user AS current_user,  
       inet_server_port() AS server_port;
```

Output:

```
database_name | current_user | server_port
```

```
-----+-----+-----
```

```
icicibank    | <your_user>  | <your_port>
```

The exact username and port depend on the PostgreSQL installation being used.

Q12. Columns of the loan Table

```
SELECT column_name,  
       data_type,  
       is_nullable  
FROM information_schema.columns
```

```
WHERE table_schema = 'public'
```

```
AND table_name = 'loan'
```

```
ORDER BY ordinal_position;
```

To find the number of columns:

```
SELECT COUNT(*) AS number_of_columns
```

```
FROM information_schema.columns
```

```
WHERE table_schema = 'public'
```

```
AND table_name = 'loan';
```

The **degree of the loan relation is equal to the number of columns in the table.**

Output:

```
column_name | data_type | is_nullable
```

```
-----+-----+-----
```

<your output will appear here>

```
number_of_columns
```

```
-----
```

<your count>

Run these statements in your icicibank database to obtain the actual output.

Q13. Number of Rows in All Nine Tables

The following PostgreSQL query lists all user tables in the public schema and counts their rows:

```
SELECT table_name,
```

```
       (xpath('/row/c/text()',
```

```
           query_to_xml(
```

```
               format('SELECT COUNT(*) AS c FROM %I.%I', table_schema, table_name),
```

```
               false,
```

```
               true,
```

```
               ''
```

```
           )
```

```
       ))[1]::text::bigint AS row_count
```

```
FROM information_schema.tables
```

```
WHERE table_schema = 'public'
```

```
AND table_type = 'BASE TABLE'
```

ORDER BY table_name;

Expected format:

table_name	row_count
------------	-----------

-----+-----

account	45
---------	----

account_txn	...
-------------	-----

branch	...
--------	-----

customer	30
----------	----

employee	...
----------	-----

fixed_deposit	...
---------------	-----

loan	...
------	-----

<remaining table>	...
-------------------	-----

<remaining table>	...
-------------------	-----

The exact counts must be obtained by running the query against the icicibank database.

Q14. List Every Foreign Key

SELECT

tc.table_name AS child_table,

kcu.column_name AS child_column,

ccu.table_name AS parent_table,

ccu.column_name AS parent_column

FROM information_schema.table_constraints AS tc

JOIN information_schema.key_column_usage AS kcu

ON tc.constraint_name = kcu.constraint_name

AND tc.table_schema = kcu.table_schema

JOIN information_schema.constraint_column_usage AS ccu

ON tc.constraint_name = ccu.constraint_name

AND tc.table_schema = ccu.table_schema

WHERE tc.constraint_type = 'FOREIGN KEY'

AND tc.table_schema = 'public'

ORDER BY tc.table_name, kcu.column_name;

Output:

child_table | child_column | parent_table | parent_column

-----+-----+-----+-----

<actual output from icicibank>

The table with a foreign key pointing to itself is the **employee table**, where manager_id references employee.employee_id. This represents the reporting hierarchy in the bank, where one employee can be the manager of another employee.

Q15. Three Largest Tables on Disk Including Indexes

SELECT

 schemaname,

 relname AS table_name,

 pg_size_pretty(

 pg_total_relation_size(relid)

) AS total_size

FROM pg_catalog.pg_statio_user_tables

ORDER BY pg_total_relation_size(relid) DESC

LIMIT 3;

Output:

schemaname | table_name | total_size

-----+-----+-----

<actual output from icicibank>

account_txn is expected to be the largest because it stores individual transaction records, and transaction volume is much higher than the number of customers, accounts or branches. It also accumulates continuously over time and may have indexes that add to its total disk usage.

Q16. Insert a New Customer and Account

First insert the customer:

INSERT INTO customer

(customer_id, name, date_of_birth, email, phone, city,

kyc_status, risk_rating, customer_since, home_branch)

VALUES

(1032, 'Latha Krishnan', DATE '1985-09-23',

'latha.krishnan@example.com', '9845099887', 'Chennai',

'VERIFIED', 'MEDIUM', DATE '2026-03-01', 5);

Insert the account:

```
INSERT INTO account
(account_id, account_number, customer_id, account_type,
balance, opened_date, interest_rate)
VALUES
(9047, 'ICIC0000009047', 1032, 'CURRENT',
500000.00, DATE '2026-03-01', 0.00);
```

Read the customer row:

```
SELECT *
FROM customer
WHERE customer_id = 1032;
```

Read the account row:

```
SELECT *
FROM account
WHERE account_id = 9047;
```

Expected customer output:

```
customer_id = 1032
name = Latha Krishnan
date_of_birth = 1985-09-23
email = latha.krishnan@example.com
phone = 9845099887
city = Chennai
kyc_status = VERIFIED
risk_rating = MEDIUM
customer_since = 2026-03-01
home_branch = 5
```

Expected account output:

```
account_id = 9047
account_number = ICIC0000009047
account_type = CURRENT
balance = 500000.00
```

opened_date = 2026-03-01

interest_rate = 0.00

Q17. Deposit 75,000 and Withdraw 20,000

Deposit 75,000:

UPDATE account

SET balance = balance + 75000

WHERE account_id = 9047;

Withdraw 20,000:

UPDATE account

SET balance = balance - 20000

WHERE account_id = 9047;

Confirm the final balance:

SELECT account_id,

account_number,

balance

FROM account

WHERE account_id = 9047;

Expected output:

account_id | account_number | balance

-----+-----+-----

9047 | ICIC0000009047 | 555000.00

Q18. Delete Customer 1032 in the Correct Order

First try to delete the customer:

DELETE FROM customer

WHERE customer_id = 1032;

Expected error if the foreign key prevents deletion:

ERROR: update or delete on table "customer" violates foreign key constraint

The exact PostgreSQL message will include the actual foreign-key constraint name in your database, so it should be copied from your terminal after running the statement.

Now delete the dependent account first:

DELETE FROM account

```
WHERE account_id = 9047;
```

Then delete the customer:

```
DELETE FROM customer
```

```
WHERE customer_id = 1032;
```

Confirm the counts:

```
SELECT COUNT(*) AS customer_count  
FROM customer;
```

```
SELECT COUNT(*) AS account_count  
FROM account;
```

Expected result:

```
customer_count
```

```
-----
```

```
30
```

```
account_count
```

```
-----
```

```
45
```

Q19. Statements That Must Be Rejected

1. Account with invalid account_type = 'FIXED':

```
INSERT INTO account
```

```
(account_id, account_number, customer_id, account_type,  
balance, opened_date, interest_rate)
```

```
VALUES
```

```
(9999, 'INVALID00009999', 1001, 'FIXED',  
10000.00, CURRENT_DATE, 5.00);
```

Expected error:

ERROR: new row violates check constraint

The exact constraint name depends on the database schema.

2. Transaction with amount -5000:

```
INSERT INTO account_txn
```

```
(transaction_id, account_id, amount)
```

VALUES

(9999, 9004, -5000);

Expected error:

ERROR: new row violates check constraint

3. Customer with kyc_status = 'IN_PROGRESS':

INSERT INTO customer

(customer_id, name, kyc_status)

VALUES

(9999, 'Test Customer', 'IN_PROGRESS');

Expected error:

ERROR: new row violates check constraint

The exact error text and constraint name must be copied from the PostgreSQL output after running each statement.

Q20. Demonstrate Isolation Using Two Sessions

SESSION A:

BEGIN;

UPDATE account

SET balance = balance + 1000

WHERE account_id = 9004;

SELECT account_id, balance

FROM account

WHERE account_id = 9004;

Output in Session A:

account_id | balance

-----+-----

9004 | <original balance + 1000>

Do not commit Session A yet.

SESSION B:

SELECT account_id, balance

FROM account

WHERE account_id = 9004;

Output in Session B:

account_id | balance

-----+-----

9004 | <original balance>

The uncommitted change made by Session A is not visible to Session B under PostgreSQL's default transaction isolation level.

Now in Session A:

ROLLBACK;

After rollback, Session A's change is removed.

Q21. Transfer 12,000 and Roll It Back

Before the transfer:

SELECT account_id, balance

FROM account

WHERE account_id IN (9004, 9010)

ORDER BY account_id;

Begin the transaction:

BEGIN;

Debit account 9004:

UPDATE account

SET balance = balance - 12000

WHERE account_id = 9004;

Credit account 9010:

UPDATE account

SET balance = balance + 12000

WHERE account_id = 9010;

Check the balances inside the transaction:

SELECT account_id, balance

FROM account

WHERE account_id IN (9004, 9010)

```
ORDER BY account_id;
```

Roll back:

```
ROLLBACK;
```

Check the balances again:

```
SELECT account_id, balance
```

```
FROM account
```

```
WHERE account_id IN (9004, 9010)
```

```
ORDER BY account_id;
```

The balances after the rollback should be exactly the same as the balances before the transfer.

Q22. PostgreSQL Configuration Values

```
SHOW shared_buffers;
```

```
SHOW work_mem;
```

```
SHOW wal_level;
```

```
SHOW fsync;
```

Or show all four together:

```
SELECT name, setting
```

```
FROM pg_settings
```

```
WHERE name IN ('shared_buffers', 'work_mem', 'wal_level', 'fsync')
```

```
ORDER BY name;
```

1. **shared_buffers:** Controls the amount of memory PostgreSQL uses for its shared buffer cache to hold database pages.
2. **work_mem:** Controls the amount of memory available to each query operation such as sorting and hashing before temporary disk files may be used.
3. **wal_level:** Controls the amount of information written to the Write-Ahead Log for recovery and features such as replication.
4. **fsync:** Controls whether PostgreSQL ensures that data and WAL changes are flushed to durable storage, which is critical for crash safety.

MODULE 1 | LEVEL 3 — ANALYSIS AND DESIGN DECISIONS

Q23. MyISAM for a UPI Product

I would not recommend MyISAM for a core UPI transaction workload because MyISAM does not provide the transaction and row-level locking characteristics required for safe financial updates. A concurrent payment workload could therefore create serious consistency and recovery risks when multiple transactions update related account data. I would use **InnoDB** if MySQL is selected, because it provides transactions, row-level locking, foreign keys and crash recovery. For a banking payment

system, transaction correctness and recovery are more important than choosing a storage engine primarily for read performance.

Q24. Account Transactions: OLTP and OLAP

One design is unlikely to serve both workloads efficiently because the branch screen requires fast, selective retrieval of transactions for one account, while the risk team's monthly totals require large scans and aggregations across millions of rows. I would keep the operational account_txn workload in an **OLTP database** with appropriate indexes and use a separate **OLAP/reporting system** populated from the operational database. The OLTP portion belongs to the database fundamentals and SQL work, while the analytical/reporting portion is the part developed through the **OLAP/data-engineering module** of the programme.

Q25. "Transfer Successful" in the Application Log

An application log saying "transfer successful" only proves that the application reached the point where it recorded that message; it does not prove that the database transaction committed successfully or that both account balances were changed correctly. A power failure, rollback, database error or network failure could occur before the transaction was committed. Evidence would be the **committed database transaction and corresponding debit and credit records**, verified by querying the account balances and transaction/audit records and confirming that the transaction identifier was successfully committed.

Q26. fsync = off on the Core Banking Server

I would not allow fsync = off on the production core banking PostgreSQL server because it can allow committed data or WAL records to be lost or corrupted after an operating-system or hardware failure. The speed improvement is not an acceptable trade-off for the durability required by financial transactions. If it is used anywhere, it should be limited to a **disposable development, testing or temporary staging environment** where data loss is acceptable and the environment is recreated from source data when necessary.

Q27. CHECK Constraint vs Application Validation

The balance ≥ 0 rule should also be enforced in the database because multiple applications, ATMs, branches, APIs and batch jobs may modify the same account, and the database provides a common protection point. Database enforcement also prevents a programming error or a new application from accidentally inserting invalid data. The application should still validate the rule for better user messages and business workflows, but it should not be the only protection. The application layer is genuinely the right place for rules that depend on user interaction, external services or complex business processes that cannot reasonably be expressed as database constraints.

Q28. Oracle vs PostgreSQL — Four Expected Differences

1. **SQL syntax and functions:** Oracle and PostgreSQL have different functions, operators and SQL-specific features, so queries may require rewriting.
2. **Data types:** Some Oracle data types and PostgreSQL data types behave differently, requiring careful mapping during migration.
3. **Transaction and configuration behaviour:** Transaction handling, isolation configuration, WAL/recovery mechanisms and database settings differ between the two systems.

4. **Procedural code and administration:** Oracle PL/SQL objects, sequences, packages, procedures, triggers and administration tools do not map directly to PostgreSQL and may require conversion or redesign.

Q29. Reply to Ticket ICICI-4417

Response to Ticket ICICI-4417

The move from spreadsheets to a relational database will give the bank one controlled place to store customer and banking information.

It will reduce duplicate copies of the same customer data across different departments.

When an address or other customer detail changes, authorised systems can work from the same updated record.

The database can also enforce rules so that invalid account, customer and transaction data is rejected.

It can control simultaneous updates so that two banking operations do not incorrectly overwrite each other.

It will also provide better support for backups, recovery, auditing and controlled access.

However, moving to a database will not automatically make incorrect data correct.

It will not prevent poor data entry, incorrect business processes or unauthorised access by itself.

The bank must therefore define clear data standards and validation rules.

Staff must follow controlled procedures for creating, updating and closing customer records.

Access must be limited according to job responsibilities, and important changes must be monitored and audited.

The bank should also maintain backups, test recovery procedures and regularly check the quality of its data.

The database provides the foundation for reliable information, but people, processes and controls are still required to use it correctly.